

Write a C program to simulate page replacement algorithms.

- a) FIFO
- b) LRU
- c) Optimal

Code:

```
#include <stdio.h>

#include <stdlib.h>

void fifo(int pages[], int n, int frames) {
    int frame[frames];

    int i, j, k = 0, page_faults = 0;

    for (i = 0; i < frames; i++) frame[i] = -1;

    printf("\n--- FIFO Page Replacement ---\n");

    for (i = 0; i < n; i++) {
        int found = 0;

        for (j = 0; j < frames; j++) {
            if (frame[j] == pages[i]) {
                found = 1;
                break;
            }
        }

        if (!found) {
            frame[k] = pages[i];
            k = (k + 1) % frames;
            page_faults++;
        }

        printf("Page %d -> [", pages[i]);

        for (j = 0; j < frames; j++) printf("%d ", frame[j]);

        printf("]\n");
    }
}
```

```
    printf("Total Page Faults (FIFO): %d\n", page_faults);  
}
```

```
void lru(int pages[], int n, int frames) {  
    int frame[frames], time[frames];  
    int i, j, page_faults = 0, t = 0;  
    for (i = 0; i < frames; i++) {  
        frame[i] = -1;  
        time[i] = 0;  
    }  
}
```

```
printf("\n--- LRU Page Replacement ---\n");
```

```
for (i = 0; i < n; i++) {  
    int found = 0;  
    for (j = 0; j < frames; j++) {  
        if (frame[j] == pages[i]) {  
            found = 1;  
            t++;  
            time[j] = t;  
            break;  
        }  
    }  
    if (!found) {  
        int min_time = time[0], pos = 0;  
        for (j = 1; j < frames; j++) {  
            if (time[j] < min_time) {  
                min_time = time[j];  
                pos = j;  
            }  
        }  
        frame[pos] = pages[i];  
    }  
}
```

```

        t++;

        time[pos] = t;

        page_faults++;
    }

    printf("Page %d -> [", pages[i]);

    for (j = 0; j < frames; j++) printf("%d ", frame[j]);

    printf("]\n");
}

printf("Total Page Faults (LRU): %d\n", page_faults);
}

```

```

void optimal(int pages[], int n, int frames) {
    int frame[frames], i, j, page_faults = 0;

    for (i = 0; i < frames; i++) frame[i] = -1;

    printf("\n--- Optimal Page Replacement ---\n");

    for (i = 0; i < n; i++) {
        int found = 0;

        for (j = 0; j < frames; j++) {
            if (frame[j] == pages[i]) {
                found = 1;
                break;
            }
        }

        if (!found) {
            int replace_index = -1, farthest = i;

            for (j = 0; j < frames; j++) {
                int k;

                for (k = i + 1; k < n; k++) {
                    if (frame[j] == pages[k]) {

```

```

        if (k > farthest) {
            farthest = k;
            replace_index = j;
        }
        break;
    }
}

if (k == n) { // never used again
    replace_index = j;
    break;
}

if (replace_index == -1) replace_index = 0;
frame[replace_index] = pages[i];
page_faults++;
}

printf("Page %d -> [", pages[i]);
for (j = 0; j < frames; j++) printf("%d ", frame[j]);
printf("]\n");
}

printf("Total Page Faults (Optimal): %d\n", page_faults);
}

```

```

int main() {
    int n, frames;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    int pages[n];

    printf("Enter page reference string: ");
    for (int i = 0; i < n; i++) scanf("%d", &pages[i]);

    printf("Enter number of frames: ");

```

```

scanf("%d", &frames);

fifo(pages, n, frames);

lru(pages, n, frames);

optimal(pages, n, frames);

return 0;

}

```

Output:

```

C:\Users\Admin\Desktop < + ~
Enter number of pages: 12
Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3

--- FIFO Page Replacement ---
Page 7 -> [7 -1 -1]
Page 0 -> [7 0 -1]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 3 1]
Page 0 -> [2 3 0]
Page 4 -> [4 3 0]
Page 2 -> [4 2 0]
Page 3 -> [4 2 3]
Page 0 -> [0 2 3]
Page 3 -> [0 2 3]
Total Page Faults (FIFO): 10

--- LRU Page Replacement ---
Page 7 -> [7 -1 -1]
Page 0 -> [7 0 -1]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 4 -> [4 0 3]
Page 2 -> [4 0 2]
Page 3 -> [4 3 2]
Page 0 -> [0 3 2]
Page 3 -> [0 3 2]
Total Page Faults (LRU): 9

--- Optimal Page Replacement ---
Page 7 -> [7 -1 -1]
Page 0 -> [0 -1 -1]
Page 1 -> [0 1 -1]
Page 2 -> [0 2 -1]
Page 0 -> [0 2 -1]
Page 3 -> [0 2 3]
Page 0 -> [0 2 3]
Page 4 -> [4 2 3]
Page 2 -> [4 2 3]
Page 3 -> [4 2 3]
Page 0 -> [0 2 3]
Page 3 -> [0 2 3]
Total Page Faults (Optimal): 7

Process returned 0 (0x0)   execution time : 28.710 s
Press any key to continue.

```